Bus Architecture

Shobhanjana Kalita,
Dept. of CSE, Tezpur University

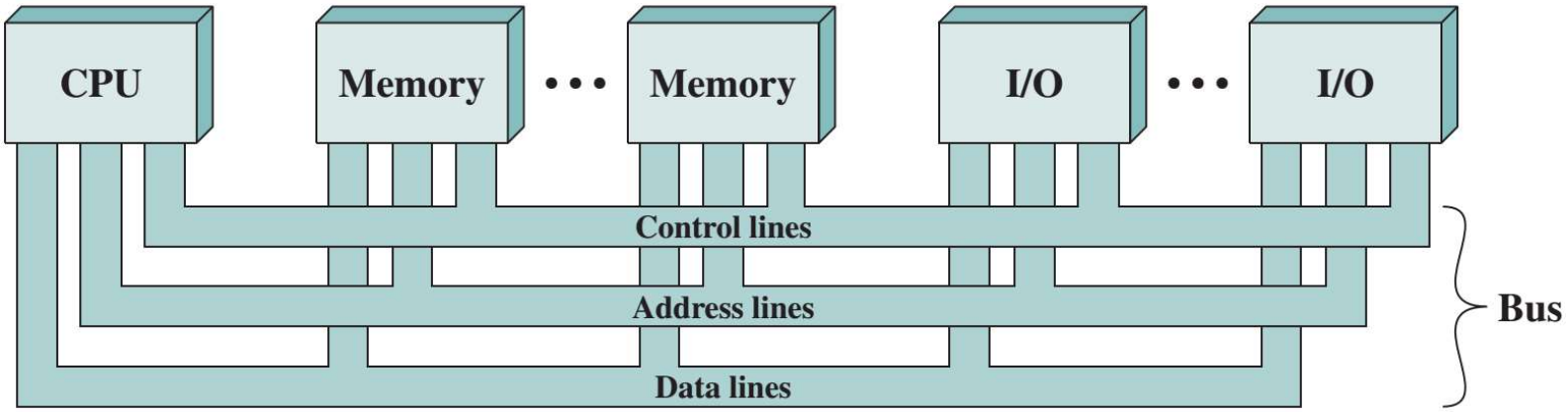
Slides courtesy: Computer Architecture and Organization, 9th Ed, W. Stallings

Bus Interconnection

- A **bus** is a communication pathway between 2 or more devices
  - A shared transmission medium
  - Multiple devices connect to the bus; a signal transmitted by one device can be received by all other devices attached to the bus
  - Only one device at a time can successfully transmit
- A bus consists of multiple communication pathways, or **lines**
  - Each line can transmit signals representing binary 1 and 0
  - Over time, a sequence of binary digits can be transmitted across a single line
  - Several lines of a bus can be used to transmit binary digits simultaneously (in parallel) – e.g. 8 bits (1 byte) over 8 bus lines

Bus Structure

- Bus connecting major computer components (processor, memory, I/O) is called a **system bus**
  - consists of 50-100s of separate lines; each line with its own particular meaning
  - lines can be classified into 3 functional groups – **data, address** and **control** lines



Bus Structure – Data bus

- **Data lines** provide a path for moving data among system modules - collectively called the **data bus**
  - may consist of 32, 64, 128, or more separate lines
  - number of lines is the **width of the data bus**
- The number of lines determines how many bits can be transferred at a time
  - The width of the data bus is a key factor in determining overall system performance
- E.g. : If the data bus is 32 bits wide and each instruction is 64 bits long, then the processor must access the memory module twice to fetch a single instruction

Bus Structure – Address bus

- **Address lines** are used to designate the source or destination of the data on the data bus
- The **width of the address bus** determines the maximum possible memory capacity of the system
- The address lines are can also used to address I/O ports.
  - Higher-order bits are used to select ***a particular module*** on the bus, and lower-order bits select ***a memory location or I/O port within the module***.
  - For example, an 8-bit address 01111111 and below might reference locations in a memory module (module 0), and address 10000000 and above refer to devices attached to an I/O module (module 1)
    - **Memory mapped I/O**

Bus Structure – Control bus

- **Control lines** are used to control the access to and the use of the data and address lines
- Controls how data and address lines are shared by all components
- Transmits command and timing information to all system modules
  - **Timing signals** indicate the validity of data and address information.
  - **Command signals** specify operations to be performed

Bus Structure – Control bus

- Typical control lines include:
  - **Memory write:** causes data on the bus to be written into the addressed location
  - **Memory read:** causes data from the addressed location to be placed on the bus
  - **I/O write:** causes data on the bus to be output to the addressed I/O port
  - **I/O read:** causes data from the addressed I/O port to be placed on the bus
  - **Transfer ACK:** indicates that data have been accepted from or placed on the bus
  - **Bus request:** indicates that a module needs to gain control of the bus
  - **Bus grant:** indicates that a requesting module has been granted control of the bus
  - **Interrupt request:** indicates that an interrupt is pending
  - **Interrupt ACK:** acknowledges that the pending interrupt has been recognized
  - **Clock:** is used to synchronize operations
  - **Reset:** initializes all modules

Bus Types

- Bus lines can be of two types: Dedicated and Multiplexed
- **Dedicated:** A bus line that is permanently assigned either to one function or to a physical subset of computer components
  - **Functional dedication:** separate dedicated address and data lines
  - **Physical dedication:** use of multiple buses, each of which connects only a subset of modules.
    - E.g: An I/O bus to interconnect all I/O modules; this bus is then connected to the main bus through some type of I/O adapter module.
    - **Advantage**: High throughput, because there is less bus contention
    - **Disadvantage:** Increased size and cost of the system

Bus Types

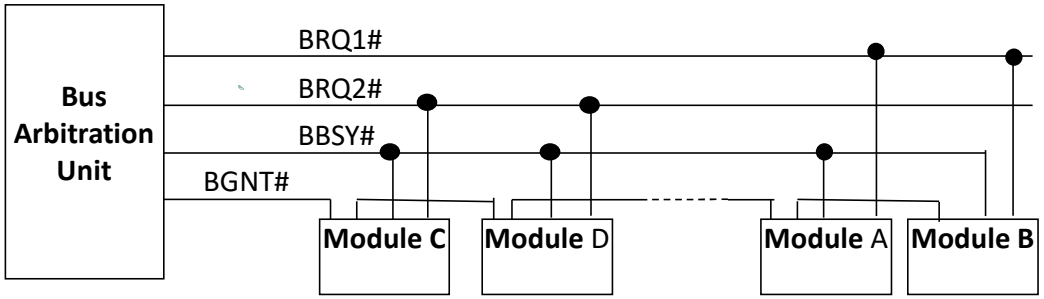
- **Multiplexed:** When the same set of bus lines is shared for multiple functions or multiple components
  - **Functional multiplexing:** same set of lines is used as an address bus as well as a data bus; use an Address Valid control line which will indicate whether an address value or data value is currently being transmitted (*time multiplexed*)
    - **Advantage:** Use of fewer lines, which saves space and cost
    - **Disadvantage:** 1. A complex circuitry is needed within each module; 2. potential reduction in performance because certain events that share the same lines cannot take place in parallel.
  - **Physical multiplexing:** when the same set of lines is shared by more than two modules (*shared bus concept*)

Bus Arbitration

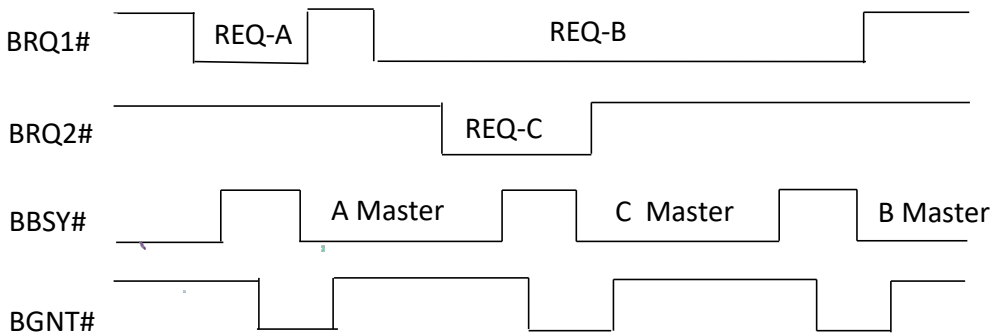
- When there is no physical dedication of bus lines (i.e. shared bus), **bus arbitration** is required to decide which module can transmit at any given time
  - Designate one device as **bus master** or **initiator** such that only the designated device may initiate a data transfer at any given point of time
- Two types of arbitration schemes: **Centralized arbitration** or **distributed arbitration**
  - *Centralized arbitration*: a single hardware device, called the **bus arbiter**, is responsible for performing arbitration or allocation of bus. It may be a separate module or part of the processor
  - *Distributed arbitration*: there is no central controller. All devices/modules participate in the selection of the next bus master

Centralized arbitration

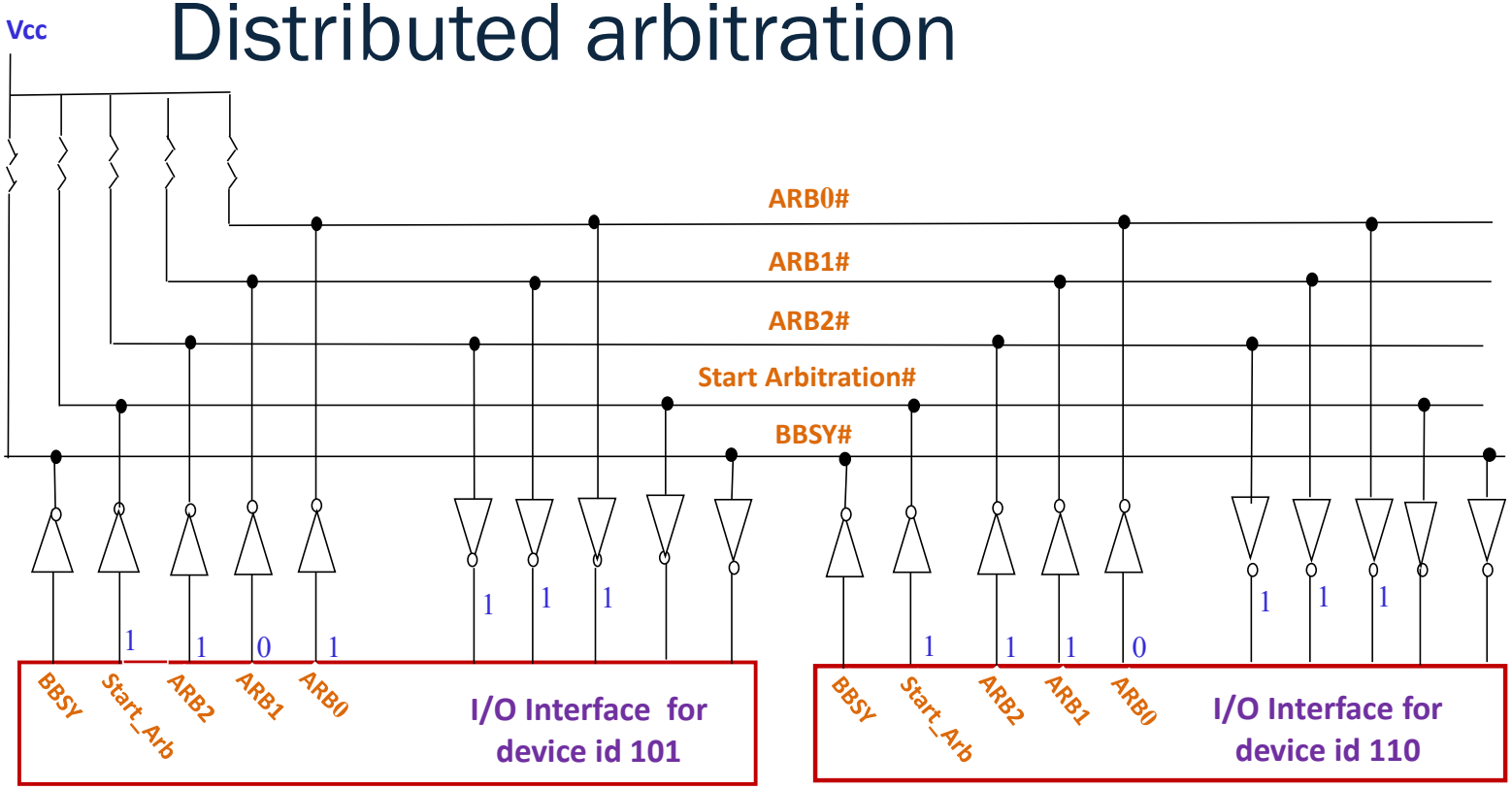
- Processor or Bus Arbitration Unit is the bus master/initiator until it grants bus mastership to one of the requesting modules.
- Modules may be organized in different priority groups and each group is connected to a different bus request line. Within a group the modules are connected in a daisy chain.
- Bus arbiter sends the bus grant (BGNT) signal through the daisy chain, ensuring higher priority module is serviced.
- The bus busy (BBSY) line is made inactive by the arbiter, and the module receiving the BGNT assumes control of the bus – i.e. it is now the bus master.



Allocation Process Timing Diagram:



Distributed arbitration

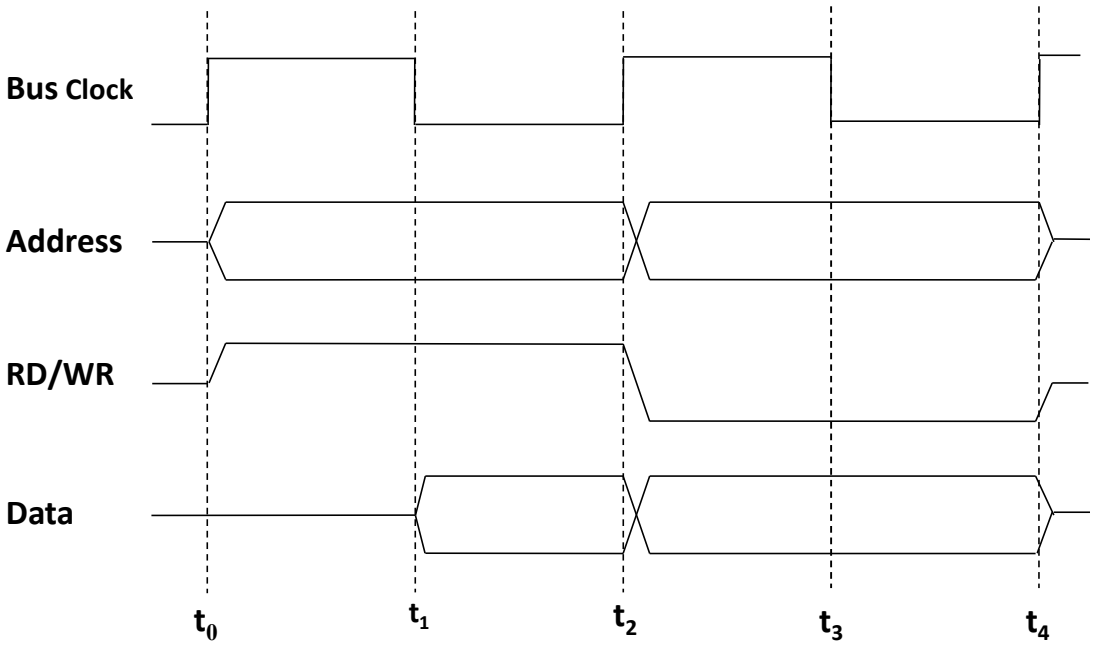


- Each device on the bus has a unique id.
- Devices requesting the bus assert Start Arbitration signal and place their own id on the arbitration lines (ARB0 through ARB2).
- If multiple devices place their ids on the lines, the OR of the signals is seen on the lines
- The devices then compare the signal seen on the ARB# lines to their own device id starting from MSB. If it sees a mismatch, it immediately disable its own drivers.
- The device that wins the arbitration is the device whose id matches
- **Distributed arbitration has higher reliability and independent of a centralized system**

Data Transfer Protocols

- There are two types of data transfer protocols depending on how the events are coordinated on the bus
  - **Synchronous data transfer:** The occurrence of events on the bus is determined by a clock
  - **Asynchronous data transfer:** The occurrence of events on the bus are not synchronized by a clock
    - Strobe control
    - Handshaking

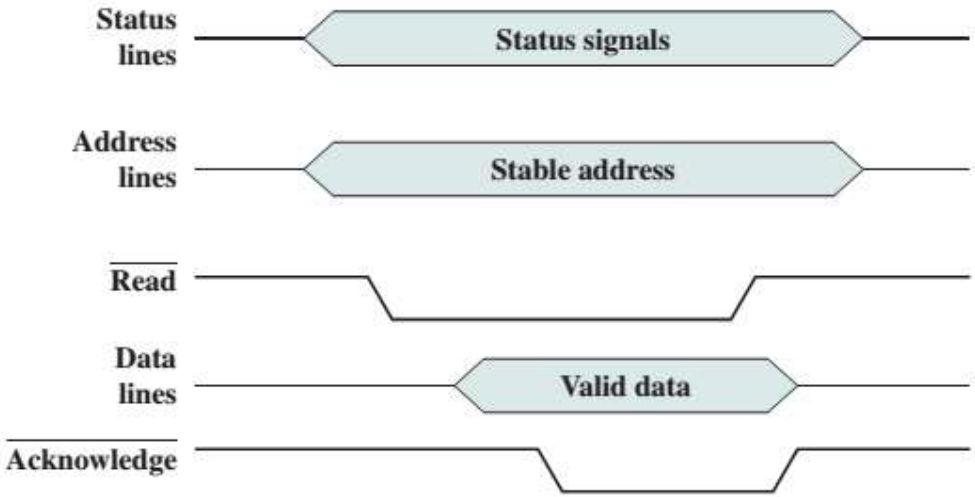
Synchronous Data Transfer



- The bus includes a clock line upon which a clock transmits a regular sequence of alternating 1s and 0s of equal duration
- A single 1–0 transmission is referred to as a *clock cycle* or *bus cycle* and defines a time slot.
- All other devices on the bus can read the clock line, and all events start at the beginning of a clock cycle

Asynchronous Data Transfer

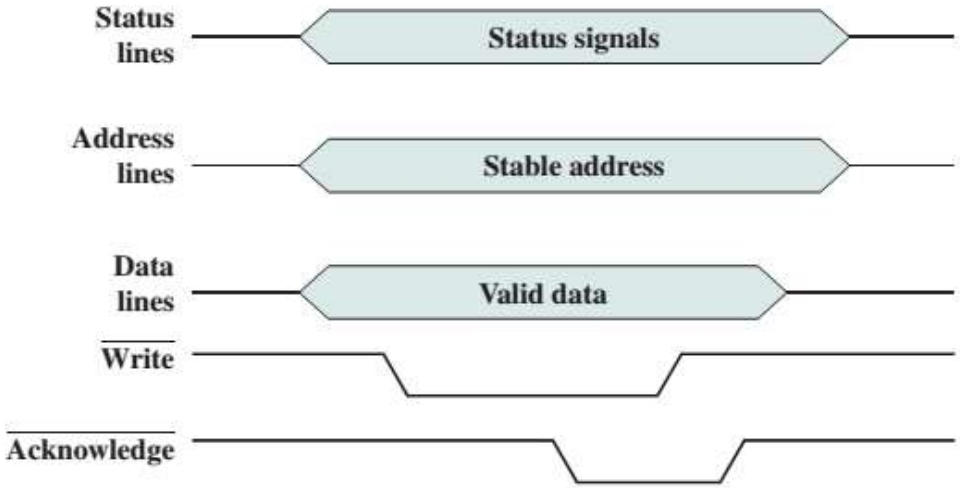
- **Asynchronous Timing:** the occurrence of one event on a bus follows and depends on the occurrence of a previous event
  - More flexible – devices are not tied to a fixed clock rate, can take advantage of advances in device performance
    - A fast machine can use the bus in between the time required for a slow device to transmit



(a) System bus read cycle

Asynchronous Data Transfer

- Asynchronous write operation:



(b) System bus write cycle

- **Handshaking protocol** ensures that both source and destination are informed about the completion of data transfer.

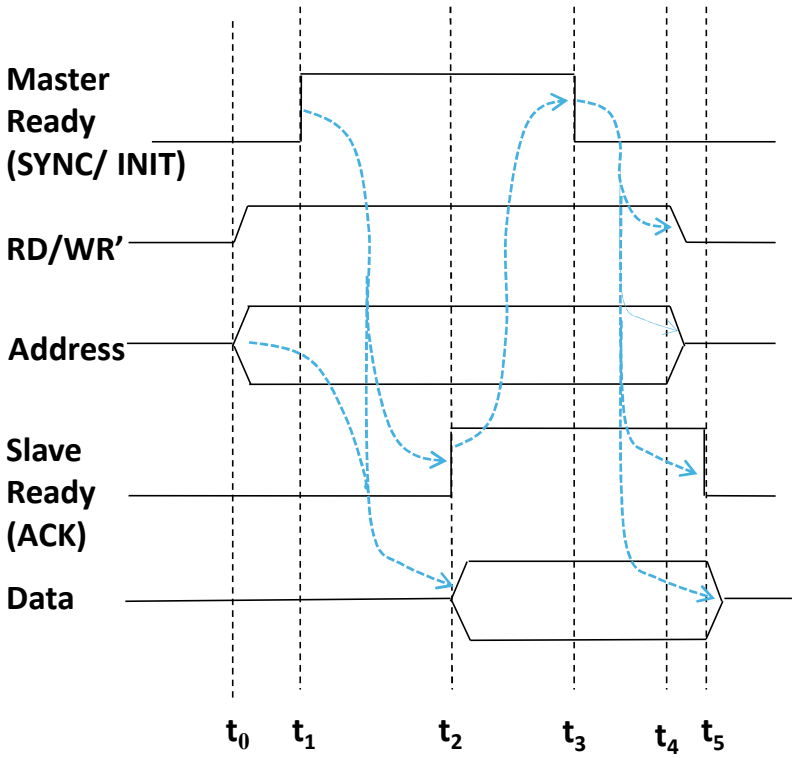
Asynchronous Data Transfer

- **Handshaking Protocol**
  - The common clock is replaced by two timing control lines – **Master-ready** and **Slave-ready***
  - The initiator places the address and command information on the bus
  - The initiator then sets the **Master-ready** line to indicate it is ready for a transaction
  - The Master-ready line causes all devices on the bus to decode the address on the bus
  - The device corresponding to the address sets the **Slave-ready** line to indicate it is ready to perform the transaction
  - The initiator on detecting the Slave-ready line gets ready for transaction
  - If it is read operation, initiator strobes for the input data

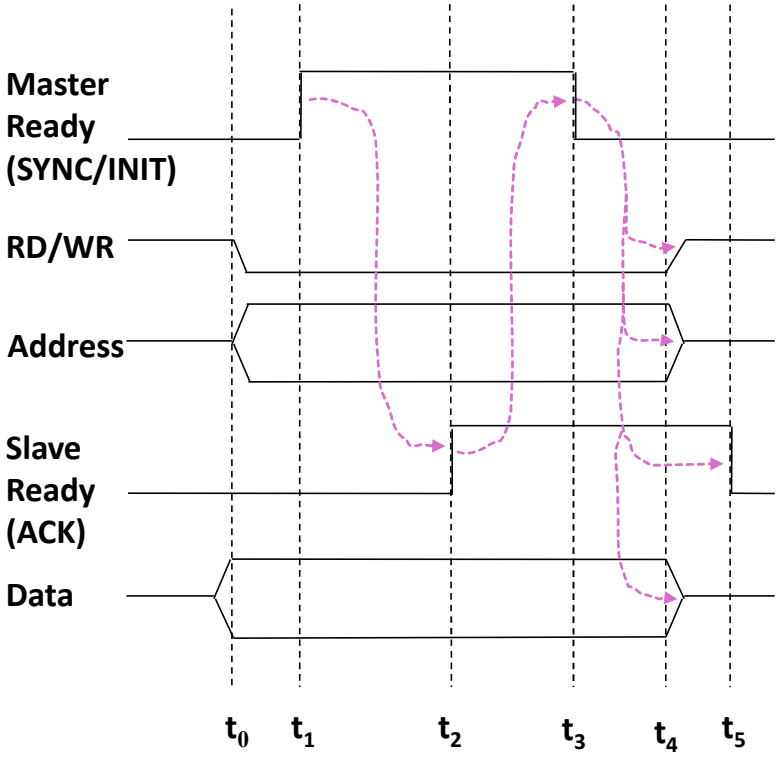
** Although the use of the terms master-slave is criticized nowadays, several earlier editions of books and reference material still carry these terms. Furthermore, other terms currently being suggested for use to replace master-slave are: requester-responder, primary-secondary, controller-device, initiator-target etc.*

Asynchronous : Handshaking

- An Input (Read) Operation)



- An Output (Write) Operation



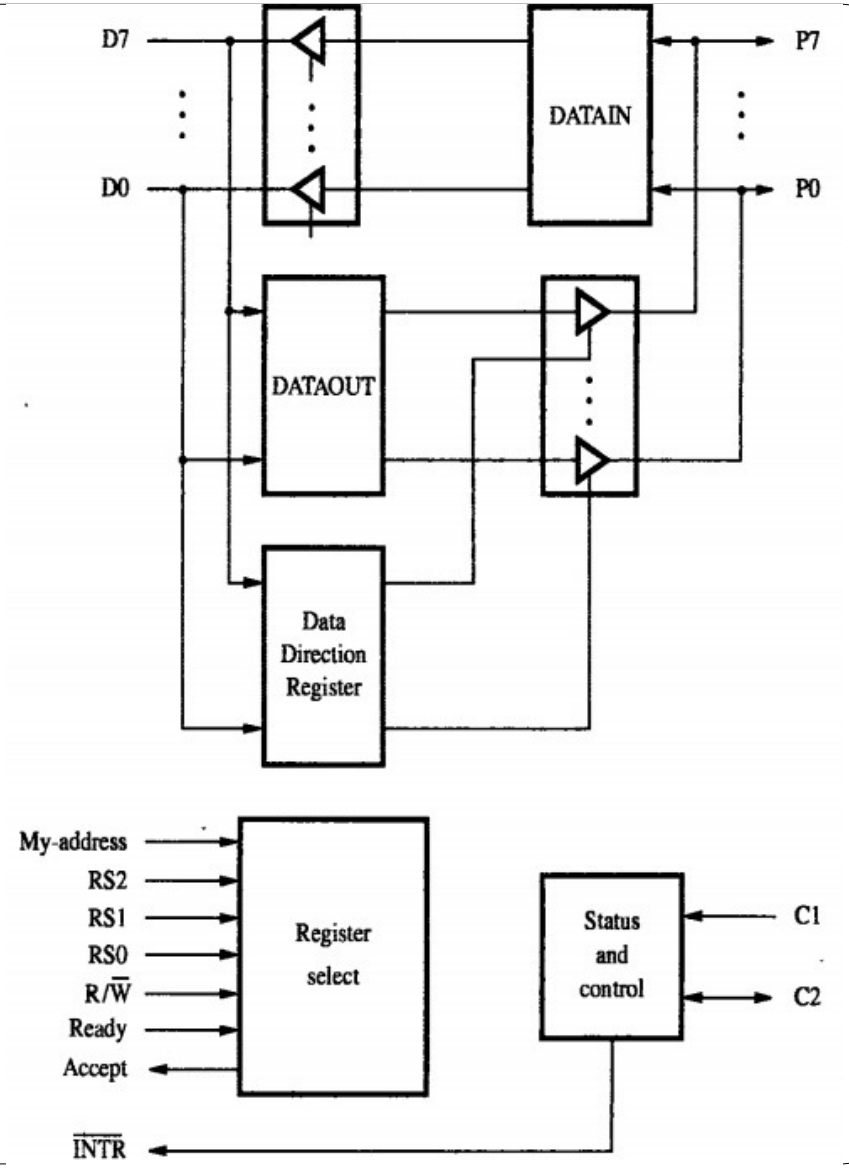
Interface Circuits

- An I/O interface consists of circuitry required to connect to an I/O device to a computer bus
  - One end of the interface consists of bus – address, data and control lines
  - Other end consists of data path and associated controls to data to I/O device – called a **port.**
- There are two types of port: **Serial Port** and **Parallel Port**

Interface Circuits : Parallel

- **Parallel port** transfers data a number of bits at the same time – usually 8 to 16
  - Connection between device and computer uses a multiple-pin connector with as many wires to carry multiple bits at a time
  - Simple; suitable when devices are physically close
  - A 25 pin connector used with printers and communication cables primarily; used initially for devices and applications that required faster data transfer

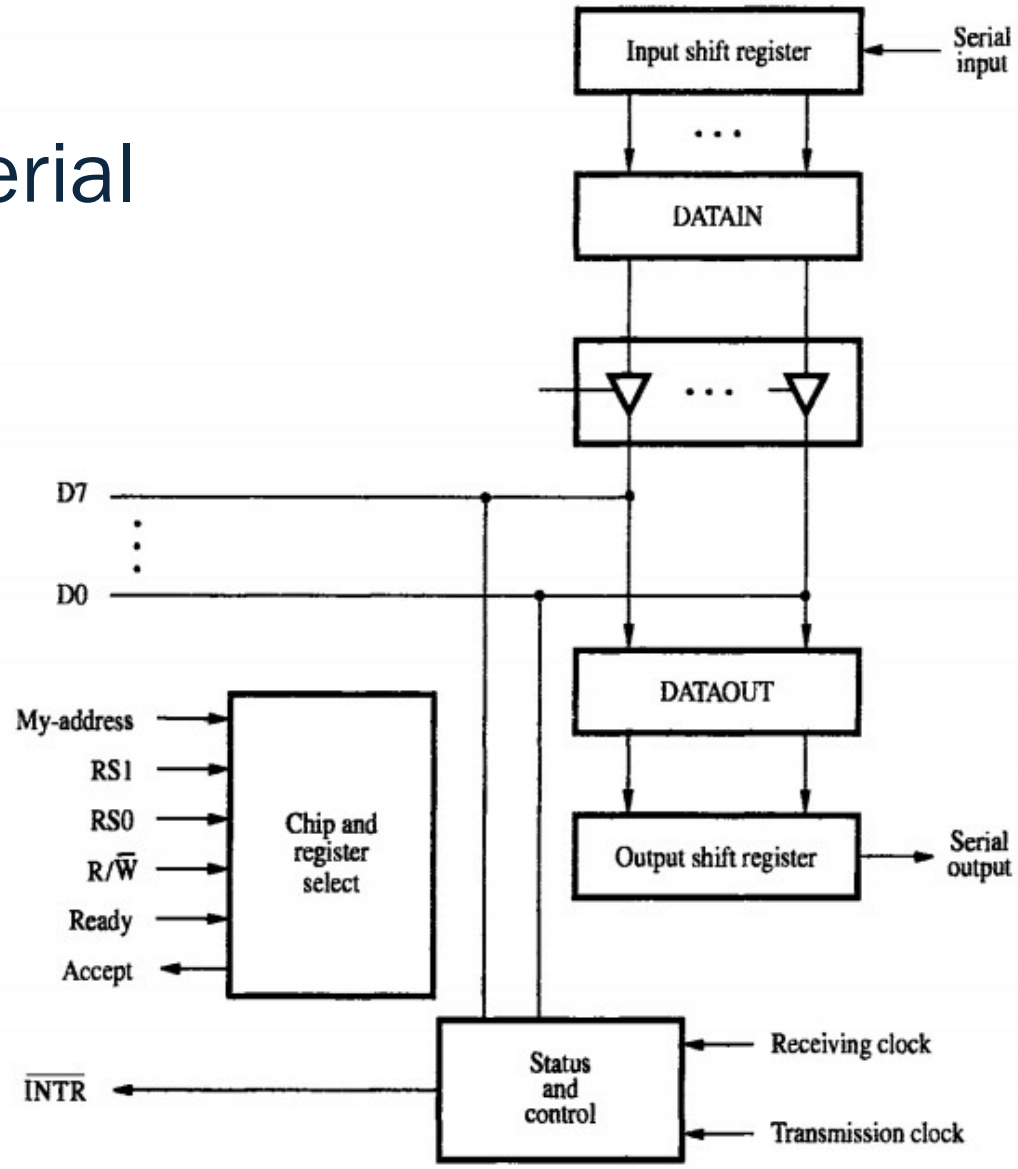
Interface Circuits : Parallel



Interface Circuits : Serial

- **Serial Port** transfers data a single bit at a time
  - On the device end communication is in a bit-serial fashion
  - On the bus side communication is in a bit-parallel fashion
  - Suitable for longer cables, because there are fewer wires
  - Less common but still used to connect a variety of devices, including mice, modems and communications cables

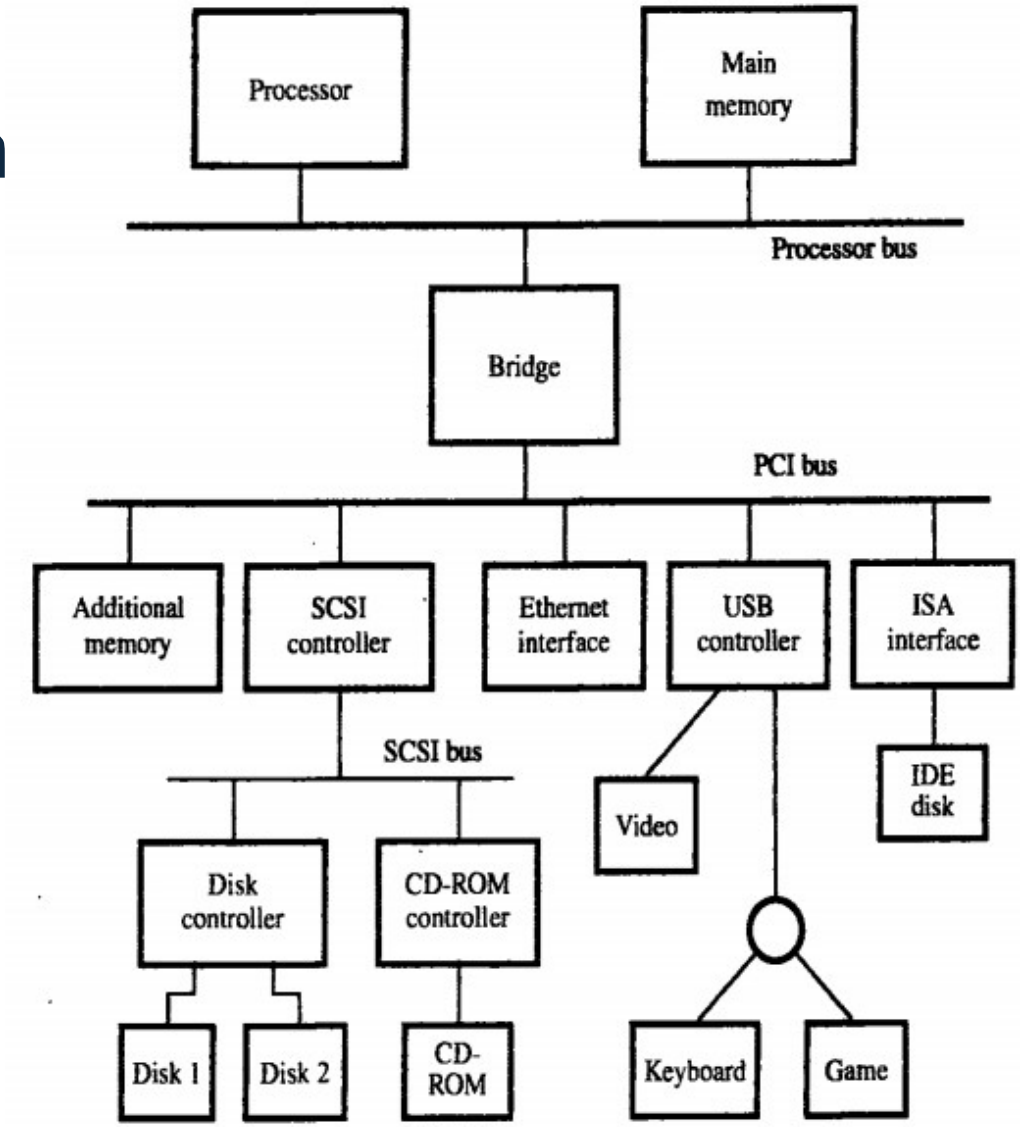
Interface Circuits : Serial



Standard I/O Interfaces

- I/O device interfaces ought to be compatible with different types of devices and computers
- A set of standard interface signals and protocols is needed
- 3 widely used bus standards are:
  - **PCI – Peripheral Component Interconnect**
  - **SCSI – Small Computer System Interface**
  - **USB – Universal Serial Bus**
- PCI defines an expansion bus to the motherboard
- SCSI and USB are used for connecting additional devices

Example connection



PCI

- **PCI** is a popular high-bandwidth, processor independent bus that can function as a peripheral bus
  - PCI delivers better system performance for high speed I/O subsystems
  - Devices connected to PCI bus appear as though they are connected directly to the processor bus
  - Used in graphic display adapters, network interface controllers, and disk controllers
- When processor request for a word, a sequence of data words starting at that address is sent
- For a write operation processor sends a memory address followed by a sequence of words

PCI

- Data Transfer
  - PCI is designed to support data transfer involving specification of an address followed by transfer of a sequence of words starting from that address location
  - Supports 3 address spaces – I/O, memory and configuration
    - But recommends memory-mapped I/O for wider compatibility
- Device Configuration
  - When a device is connected – it needs to be configured i.e. the software communicating with it needs to know its address and characteristics (transmission speed etc.)
  - PCI incorporates a small ROM with each I/O interface that stores configuration information (configuration address space)
  - PCI initialization software reads these ROMs when system is powered up – relieves user to be involved in the configuration

SCSI

- Devices connected via SCSI are not part of the processor address space
- SCSI bus is connected to the processor bus via a SCSI controller
- SCSI controller uses DMA to transfer packets from the main memory to device or vice versa
  - **Packets** may contain a block of data, commands from the processor to the device, or status information about the device
- SCSI controller can be one of two types – *initiator* or *target*
  - *Initiator* selects a target to send commands: controller on the processor side
  - *Target* carries out commands received from the initiator: controller on the device side

SCSI

- Initiator establishes a logical connection with intended target
- Once established, it can be suspended or restored as required to transfer bursts of data – while suspended bus can be used by other devices
  - Overlapping of data transfers – key feature of SCSI bus that leads to high performance
- Data transfer
  - Initiator SCSI controller requests for bus
  - On winning arbitration selects target for communication
  - Transfers control of bus to the target controller
  - Target controller starts data transfer operation to receive command from the initiator
  - Target responds with status information and data transfer operation

USB

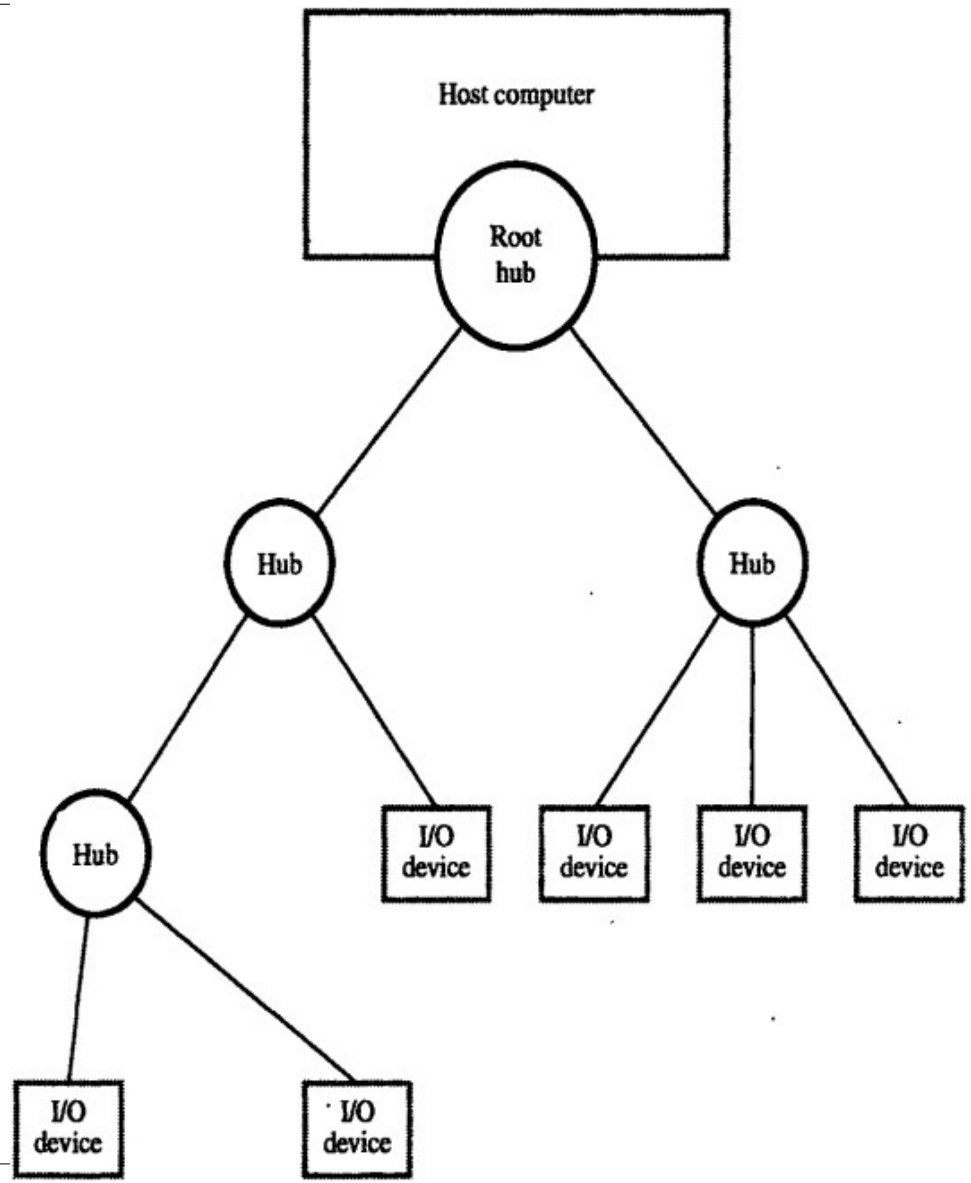
- USB (Universal Serial Bus) is a simple low cost mechanism to connect a variety of I/O devices
- Supports two operation speeds: low speed (1.5 Mbps) and full speed (12 Mbps)
  - USB 2.0 further supports High Speed (480 Mbps)
  - USB 3.0 further supports Super Speed (4.8 Gbps)
- Designed to meet following **key objectives**
  - Provide a simple, low-cost, easy-to-use interconnection system that overcomes difficulties due to limited number of I/O ports
  - Accommodate a wide range of data transfer characteristics for I/O devices – including telephone & internet connection
  - Enhance use experience through **Plug & Play** mode of operation

USB

- Port-limitation
  - Serial and parallel ports provide a general purpose point of connection – but only a few such ports on a computer
  - To add new ports physically open up and install new interfaces
  - USB makes it possible to add a wide range of device without installing new interfaces
- Device Characteristics
  - Asynchronous and low speed – events generated are not synchronized with computer – keyboard, mouse, game controllers
  - Isochronous and high speed – successive events are separated by equal periods of time – microphones (A to D converter)

USB

- Plug and play
  - Means that a new device may be connected at any time while the system is running
  - System detects the existence of this new device automatically,
    - identifies appropriate device driver software and any other facilities needed for the device
    - Establishes appropriate addresses and logical connections for communication
- USB Architecture
  - Serial transmission – low cost and flexible
  - Tree structure – each node of the tree has a device called **hub**
  - Root hub connects entire tree to host computer
  - Leaves of the tree are the I/O devices being serviced - functions

USB

- Tree structure enables many devices to be connected including other hubs while using point-to-point serial links
  - Message sent by host device is sent to all devices but only the addressed device responds to the message
- Root hub appears as single device with one address on the processor bus